

Build a RAG document store on Azure Cosmos DB for NoSQL

In this exercise, you create an Azure Cosmos DB for NoSQL database that serves as a document store for retrieval-augmented generation (RAG) applications. The database stores chunked documents with metadata that an AI application can retrieve to provide context to language models. You design a schema optimized for document retrieval, build Python functions that store and query document chunks, and test the complete workflow using a Flask web application. This pattern provides a foundation for building AI applications that ground language model responses in your organization's documents.

Tasks performed in this exercise:

- Download project starter files and configure the deployment script
- Deploy an Azure Cosmos DB for NoSQL account with a database and container
- Build Python functions for storing and retrieving document chunks
- Test the RAG functions using a Flask web application
- Query document context using the Cosmos DB SQL API

This exercise takes approximately **30** minutes to complete.

Before you start

To complete the exercise, you need:

- An Azure subscription with the permissions to deploy the necessary Azure services. If you don't already have one, you can [sign up for one](#).
- [Visual Studio Code](#) on one of the [supported platforms](#).
- The latest version of the [Azure CLI](#).
- [Python 3.12](#) or greater.

Download project starter files and deploy Azure services

In this section you download the project starter files and use a script to deploy the necessary services to your Azure subscription. The Cosmos DB account deployment takes a few minutes to complete.

1. Open a browser and enter the following URL to download the starter file. The file will be saved in your default download location.

CodeCopy

```
https://github.com/MicrosoftLearning/mslearn-azure-ai/raw/main/downloads/python/cosmosdb-build-query-python.zip
```
2. Copy, or move, the file to a location in your system where you want to work on the project. Then unzip the file into a folder.
3. Launch Visual Studio Code (VS Code) and select **File > Open Folder...** in the menu, then choose the folder containing the project files.
4. Open the *azdeploy.py* deployment script and change the two values at the top of the script to meet your needs, then save your changes. **Note:** Do not change anything else in the script.

CodeCopy

```
"<your-resource-group-name>" # Resource Group name
"<your-azure-region>" # Azure region for the resources
```

5. In the menu bar select **Terminal > New Terminal** to open a terminal window in VS Code.
6. Run the following command to login to your Azure account. Answer the prompts to select your Azure account and subscription for the exercise.

CodeCopy

```
az login
```

7. Run the following command to ensure your subscription has the necessary resource provider for the exercise.

CodeCopy

```
az provider register --namespace Microsoft.DocumentDB
```

Create resources in Azure

In this section you run the deployment script to deploy the Cosmos DB account.

1. Make sure you are in the root directory of the project and run the following command in the terminal to launch the deployment script.

CodeCopy

```
python azdeploy.py
```

2. When the script menu appears, enter **1** to launch the **Create Cosmos DB account** option. This creates the Cosmos DB for NoSQL account with a database and container. **Note:** Deployment can take 5-10 minutes to complete.

!

IMPORTANT: Leave the terminal running the deployment open for the duration of the exercise. You can move on to the next section of the exercise while the deployment continues in the terminal.

Complete the RAG document functions

In this section you complete the *rag_functions.py* file by adding functions that an AI application can call to store and retrieve document chunks. These functions serve as the application's interface to the document store. The app you run later in this exercise imports these functions to demonstrate how an AI application would use them.

1. Open the *client/rag_functions.py* file in VS Code.

!

Tip: To maintain proper code indentation, paste the code flush with the left margin (column 1), select all of the pasted lines, and press **Tab** to align the block with the **BEGIN / END** markers. Press **Shift+Tab** to outdent if needed.

1. Search for the **BEGIN STORE DOCUMENT CHUNK FUNCTION** comment and add the following code directly after the comment. This function stores a document chunk with its metadata, using upsert to handle both inserts and updates.

CodeCopy

```

def store_document_chunk(
    document_id: str,
    chunk_id: str,
    content: str,
    metadata: dict = None,
    embedding: list = None
) -> dict:
    """Store a document chunk with metadata and optional embedding placeholder."""
    container = get_container()

    # Build the document structure following our RAG schema
    # The 'id' field is required by Cosmos DB and must be unique within the
    partition
    # The 'documentId' field is our partition key – chunks from the same source
    document
    # are stored together for efficient retrieval
    chunk = {
        "id": chunk_id,
        "documentId": document_id,
        "content": content,
        "metadata": metadata or {},
        "embedding": embedding or [], # Placeholder for vector embeddings
        "createdAt": datetime.utcnow().isoformat(),
        "chunkIndex": metadata.get("chunkIndex", 0) if metadata else 0
    }

    # upsert_item inserts if new, updates if exists (based on id + partition key)
    # This is idempotent – safe to call multiple times with the same data
    response = container.upsert_item(body=chunk)

    # Request Units (RUs) measure the cost of database operations in Cosmos DB
    # Tracking RU consumption helps optimize queries and estimate costs
    ru_charge = response.get_response_headers()['x-ms-request-charge']

    return {
        "chunk_id": chunk_id,
        "document_id": document_id,
        "ru_charge": float(ru_charge)
    }

```

2. Search for the **BEGIN GET CHUNKS BY DOCUMENT FUNCTION** comment and add the following code directly after the comment. This function retrieves all chunks for a specific document, ordered by chunk index for sequential reading.

Code

 Copy

[Before you start](#)

Download project starter files and deploy Azure services

[Complete the RAG document functions](#)

[Complete the Azure resource deployment](#)

[Understand the RAG document schema](#)

[Test the RAG functions with the Flask app](#)

[Query document context](#)

[Summary](#)

[Clean up resources](#)

[Troubleshooting](#)

```
def get_chunks_by_document(document_id: str, limit: int = 100) -> list:
    """Retrieve all chunks for a specific document, ordered by chunk index."""
    container = get_container()

    # SQL query using parameterized values (@documentId, @limit) to prevent
    injection
    # The 'c' alias represents each document in the container
    query = """
        SELECT c.id, c.content, c.metadata, c.chunkIndex, c.createdAt
        FROM c
        WHERE c.documentId = @documentId
        ORDER BY c.chunkIndex
        OFFSET 0 LIMIT @limit
    """

    # Single-partition query: providing partition_key limits the query to one
    partition
    # This is more efficient than cross-partition queries because Cosmos DB only
    # needs to read from one physical partition instead of fanning out to all
    partitions
    items = container.query_items(
        query=query,
        parameters=[
            {"name": "@documentId", "value": document_id},
            {"name": "@limit", "value": limit}
        ],
        partition_key=document_id # Scopes query to a single partition
    )

    # Transform Cosmos DB items into a consistent response format
    return [
        {
            "chunk_id": item["id"],
            "content": item["content"],
            "metadata": item["metadata"],
            "chunk_index": item["chunkIndex"],
            "created_at": item["createdAt"]
        }
        for item in items
    ]
```

3. Search for the **BEGIN SEARCH CHUNKS BY METADATA FUNCTION** comment and add the following code directly after the comment. This function searches for chunks across documents using metadata filters, which is useful for finding relevant context based on tags, categories, or other attributes.

Code

Copy

```

def search_chunks_by_metadata(
    filters: dict,
    limit: int = 10
) -> list:
    """Search for chunks across documents using metadata filters."""
    container = get_container()

    # Build WHERE clauses dynamically based on provided filters
    # This allows flexible querying by any combination of metadata fields
    where_clauses = []
    parameters = []

    if "source" in filters and filters["source"]:
        where_clauses.append("c.metadata.source = @source")
        parameters.append({"name": "@source", "value": filters["source"]})

    if "category" in filters and filters["category"]:
        where_clauses.append("c.metadata.category = @category")
        parameters.append({"name": "@category", "value": filters["category"]})

    if "tags" in filters and filters["tags"]:
        # ARRAY_CONTAINS checks if a value exists within an array field
        # This is useful for searching tags, keywords, or other list-based metadata
        where_clauses.append("ARRAY_CONTAINS(c.metadata.tags, @tag)")
        parameters.append({"name": "@tag", "value": filters["tags"][0]})

    # Default to "1=1" (always true) if no filters provided
    where_clause = " AND ".join(where_clauses) if where_clauses else "1=1"
    parameters.append({"name": "@limit", "value": limit})

    query = f"""
        SELECT c.id, c.documentId, c.content, c.metadata, c.chunkIndex
        FROM c
        WHERE {where_clause}
        OFFSET 0 LIMIT @limit
    """

    # Cross-partition query: searches across ALL partitions in the container
    # Required when you don't know which partition contains the data you need
    # More expensive than single-partition queries but necessary for metadata
    searches
    items = container.query_items(
        query=query,
        parameters=parameters,
        enable_cross_partition_query=True # Fan out to all partitions
    )

    return [
        {
            "chunk_id": item["id"],
            "document_id": item["documentId"],
            "content": item["content"],
            "metadata": item["metadata"],
            "chunk_index": item["chunkIndex"]
        }
        for item in items
    ]

```

4. Search for the **BEGIN GET CHUNK BY ID FUNCTION** comment and add the following code directly after the comment. This function performs an efficient point read to retrieve a specific chunk by its ID and document ID.

CodeCopy

```
def get_chunk_by_id(document_id: str, chunk_id: str) -> dict:
    """Retrieve a specific chunk using a point read (most efficient)."""
    container = get_container()

    try:
        # Point read: the most efficient Cosmos DB operation
        # By providing both the item ID and partition key, Cosmos DB can go
        # directly to the exact location of the document without any query execution
        # This results in the lowest latency and RU cost (typically 1 RU for small
docs)
        item = container.read_item(
            item=chunk_id,          # The unique ID within the partition
            partition_key=document_id # The partition where this item lives
        )
        return {
            "chunk_id": item["id"],
            "document_id": item["documentId"],
            "content": item["content"],
            "metadata": item["metadata"],
            "chunk_index": item["chunkIndex"],
            "created_at": item["createdAt"],
            "embedding": item.get("embedding", [])
        }
    except exceptions.CosmosResourceNotFoundError:
        # Return None if the item doesn't exist rather than raising an exception
        # This allows the caller to handle missing items gracefully
        return None
```

5. Save your changes to the *rag_functions.py* file.
6. Take a few minutes to review all of the code in the app.

Next, you finalize the Azure resource deployment.

Complete the Azure resource deployment

In this section you return to the deployment script to configure Entra ID access and retrieve the connection information for the Cosmos DB account.

1. When the **Create Cosmos DB account** operation has completed, enter **2** to launch the **Configure Entra ID access** option. This assigns your user account the necessary role to access the Cosmos DB data plane.
2. Enter **3** to launch the **Check deployment status** option. This verifies all resources are ready.
3. Enter **4** to launch the **Retrieve connection info** option. This creates a file with the necessary environment variables.
4. Enter **5** to exit the deployment script.
5. Run the following command to load the environment variables into your terminal session from the file created in a previous step.

Bash

CodeCopy

```
source .env
```

PowerShell

CodeCopy

```
. .\env.ps1
```

⚠ **Note:** Keep the terminal open. If you close it and create a new terminal, you might need to run the command to create the environment variable again.

Next, you explore the document schema used for the RAG document store.

Understand the RAG document schema

In this section you learn about the document schema designed for RAG applications. Unlike relational databases, Cosmos DB for NoSQL uses a flexible JSON document model. The container was created with **documentId** as the partition key, which groups all chunks from the same source document together for efficient retrieval.

The document schema for each chunk includes:

Field	Description
id	Unique identifier for the chunk (required by Cosmos DB)
documentId	Source document identifier (partition key)
content	The actual text content of the chunk
metadata	Flexible object for source, category, tags, and custom attributes
embedding	Array placeholder for vector embeddings (used in vector search scenarios)
chunkIndex	Position of the chunk within the source document
createdAt	Timestamp for when the chunk was stored

This schema supports common RAG patterns:

- **Point reads:** Retrieve a specific chunk by ID and document ID (lowest latency)
- **Single-partition queries:** Get all chunks for a document efficiently
- **Cross-partition queries:** Search across documents by metadata
- **Vector search:** When combined with vector indexing

Test the RAG functions with the Flask app

In this section you start the Flask web application and use its interface to test the RAG functions you created. The app provides a visual way to load data, run tests, query chunks, and execute custom SQL queries.

1. Run the following command to navigate to the *client* directory.

CodeCopy

```
cd client
```


2. Run the following command to create a virtual environment for the Flask app. Depending on your environment the command might be **python** or **python3**.

Code	Copy
<pre>python -m venv .venv</pre>	

3. Run the following command to activate the Python environment. **Note:** On Linux/macOS, use the Bash command. On Windows, use the PowerShell command. If using Git Bash on Windows, use **source .venv/Scripts/activate**.

Bash

Code	Copy
<pre>source .venv/bin/activate</pre>	

PowerShell

Code	Copy
<pre>.\.venv\Scripts\Activate.ps1</pre>	

4. Run the following command to install the Python dependencies for the app. This installs the **flask** and **azure-cosmos** libraries.

Code	Copy
<pre>pip install -r requirements.txt</pre>	

5. Run the following command to start the Flask application.

Code	Copy
<pre>python app.py</pre>	

6. Open a browser and navigate to `http://127.0.0.1:5000` to view the application.

Load sample data

In this section you use the app to load sample document chunks into the Cosmos DB container. The app calls the **store_document_chunk()** function you created in *rag_functions.py* to insert each chunk.

- 1. In the **Load Sample Data** section, select **Load Sample Chunks**. This inserts 12 sample chunks across four documents, representing content from fictional Azure documentation articles.
- 2. Verify that the success message appears in the **Results** section showing the number of chunks loaded and the total RU (Request Unit) charge.

Run test workflow

In this section you run automated tests that verify the RAG functions you created in *rag_functions.py* work correctly.

- 1. In the **Run Test Workflow** section, select **Run Tests**. This executes five tests that exercise each function.
- 2. Review the test results in the **Results** panel. Each test should show a **passed** status:
 - Store document chunks

- Get chunks by document ID
- Search by category
- Search by tag
- Point read by ID

Get chunks by document

In this section you retrieve all chunks for a specific document. The app calls the **get_chunks_by_document()** function you created in *rag_functions.py*.

1. In the **Get Chunks by Document** section, select a document from the dropdown (for example, **doc-azure-overview**).
2. Select **Get Chunks** to retrieve all chunks for that document.
3. Review the results showing the chunks ordered by their index, along with their content and metadata tags.

Search by metadata

In this section you search for chunks across all documents using metadata filters. The app calls the **search_chunks_by_metadata()** function you created in *rag_functions.py*. You observe how combining filters narrows the results.

1. In the **Search by Metadata** section, select **ai-applications** from the **Category** dropdown. Leave the **Tag** field empty.
2. Select **Search** to find all chunks with that category.
3. Review the results in the **Results** panel. You should see 4 chunks returned, each with different tags such as **rag**, **embeddings**, **chunking**, and **metadata**.
4. Now add a tag filter to narrow the results. Enter **embeddings** in the **Tag** field and select **Search** again.
5. Notice that fewer results are returned - only chunks that match both the **ai-applications** category and contain the **embeddings** tag. This demonstrates how combining metadata filters helps RAG applications retrieve more targeted context.

Query document context

In this section you practice writing SQL queries against the Cosmos DB container using the Query Explorer. These queries demonstrate patterns that RAG applications commonly use to retrieve document context.

1. In the **Query Explorer** section, enter the following query in the **SQL Query** field to find all chunks for a specific document. This query retrieves chunks ordered by their index for sequential reading.

Code

 Copy

```
SELECT c.id, c.chunkIndex, c.content, c.metadata
FROM c
WHERE c.documentId = 'doc-azure-overview'
ORDER BY c.chunkIndex
```

2. Select **Execute Query** and review the results.
3. Enter the following query in the **SQL Query** field to search for chunks with a specific category across all documents. This demonstrates a cross-partition query that searches metadata.

Code

 Copy

```
SELECT c.documentId, c.id, c.content, c.metadata.category
FROM c
WHERE c.metadata.category = 'cloud-services'
```

- 4. Select **Execute Query** and review the results.
- 5. Enter the following query in the **SQL Query** field to view the documents stored in the container along with their category and source metadata. This helps understand what content is available for RAG retrieval.

CodeCopy

```
SELECT DISTINCT c.documentId, c.metadata.category, c.metadata.source
FROM c
```

- 6. Select **Execute Query** and review the results.
- 7. Enter the following query in the **SQL Query** field to find chunks that contain a specific tag in their metadata. This demonstrates searching within arrays using **ARRAY_CONTAINS**.

CodeCopy

```
SELECT c.documentId, c.id, c.content, c.metadata.tags
FROM c
WHERE ARRAY_CONTAINS(c.metadata.tags, 'compute')
```

- 8. Select **Execute Query** and review the results.
- 9. Return to the terminal and press **Ctrl+C** to stop the Flask application.

Summary

In this exercise, you built a Cosmos DB-based document store for RAG applications. You deployed an Azure Cosmos DB for NoSQL account with a database and container optimized for document retrieval patterns. You created Python functions that store document chunks with metadata, retrieve chunks by document ID, search across documents using metadata filters, and perform efficient point reads. You tested the workflow using a Flask web application that exercised each function, then queried the stored data using the Cosmos DB SQL API. This pattern enables AI applications to store chunked documents and retrieve relevant context to ground language model responses.

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

- 1. Run the following command in the VS Code terminal to delete the resource group, and all resources in the group. Replace **<rg-name>** with the name you chose earlier in the exercise. The command will launch a background task in Azure to delete the resource group.

CodeCopy

```
az group delete --name <rg-name> --no-wait --yes
```

CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.

Troubleshooting

If you encounter issues during this exercise, try these steps:

Flask app fails to start

- Ensure Python virtual environment is activated (you should see **(.venv)** in your terminal prompt)
- Ensure dependencies are installed: **pip install -r requirements.txt**
- Ensure environment variables are loaded by running **source .env** (Bash) or **..env.ps1** (PowerShell)
- Ensure you are in the *client* directory when running **python app.py**

Authentication or access denied errors

- Ensure Entra ID access was configured by running the deployment script option **2**
- Verify your user has both the **Contributor** role and the **Cosmos DB Built-in Data Contributor** role
- Ensure **COSMOS_ENDPOINT** is set correctly in your terminal session

Cosmos DB operations fail

- Verify the Cosmos DB account is ready by running the deployment script option **3**
- Ensure the database and container were created during deployment
- Check that the container uses **/documentId** as the partition key

Environment variable issues

- Ensure the *.env* file was created by running the deployment script option **4**
- Run **source .env** (Bash) or **..env.ps1** (PowerShell) after creating a new terminal
- Verify variables are set by running **echo \$COSMOS_ENDPOINT** (Bash) or **\$env:COSMOS_ENDPOINT** (PowerShell)

Python venv activation issues

- On Linux/macOS, use: **source .venv/bin/activate**
- On Windows PowerShell, use: **.\venv\Scripts\Activate.ps1**
- If **activate** script is missing, reinstall **python3-venv** package and recreate the venv